6. If the parameter is a known service type from the dependency injection container, bind by injecting the service from the container.
7. Finally, bind to the body by deserializing from JSON.

The minimal API infrastructure follows this sequence for every parameter in a handler and stops at the first matching binding source.

WARNING If binding fails for the entry, and the parameter isn't optional, the request fails with a `400 Bad Request` response. The minimal API doesn't try another binding source after one source fails.

Remembering this sequence of binding sources is one of the hardest things about minimal APIs to get your head around. If you're struggling to work out why a request isn't working as you expect, be sure to come back and check this sequence. I once had a parameter that wasn't binding to a route parameter, despite its having a `TryParse` method. When I checked the sequence, I realized that it also had a `BindAsync` method that was taking precedence!

7.9 Simplifying handlers with AsParameters

Before we move on, we'll take a quick look at a .NET 7 feature for minimal APIs that can simplify some endpoint handlers: the `[AsParameters]` attribute. Consider the following `GET` endpoint, which binds to a route value, a header value, and some query values:

```
app.MapGet("/category/{id}", (int id, int page, [FromHeader(Name = "sort")]
➥ bool? sortAsc, [FromQuery(Name = "q")] string search) => { });
```